

The current OpenCog implementation

While present implementation requires work on scalability, the existing basic system is capable of the following:

- AtomSpace can be accessed using C++, Scheme, Python, Haskell, a Web interface (REST) or ZeroMQ. Low-level access via C++ is about 40 times faster than access with Scheme or Python. Thus, Scheme and Python are better suited for invoking high-level functions (such as pattern matcher, which can take a fair amount of time to search large AtomSpaces.) REST and ZeroMQ interfaces are another order of magnitude slower than Scheme/Python interfaces and are, thus, usable only for casual inspection.
- Access to AtomSpace via C++ is completely thread-safe. Access via Scheme is thread-safe, but seems to somehow serialise to only one thread, presumably due to some currently-unknown design bug/feature. Python interface is not thread-safe.
- AtomSpace contents can be saved/restored as string s-expressions (that is, Scheme), and in an SQL database.
- SQL storage backend allows a weak form of dynamic, on-demand saving and loading of hypergraphs. Contents of AtomSpace can be bulk-saved/restored. In addition, individual atoms can be saved/restored on request. This is intentional; forcing all atoms to be automatically mirrored to the database would require a large CPU overhead, especially for workloads that rapidly create and destroy a lot of transient atoms, or modify truth values at a rapid clip. Store-back queues for SQL

database are fully-parallel and asynchronous; the current design uses four store-back threads by default.

- Multiple AtomSpaces on multiple network-connected servers can communicate via SQL backend. That is, these can share common atoms. User algorithms must explicitly push atoms to the database in order for these to become visible to other AtomSpaces. This is intentional; trying to maintain constant synchronisation across multiple AtomSpaces would be extremely CPU-intensive, especially since many workloads create and destroy atoms at a rapid pace, and there is no need to share such atoms.
- A generic pattern matcher has been implemented. Queries may be specified as hypergraphs themselves, using SatisfactionLink, BindLink and GetLink. Procedures may be triggered using ExecutionOutputLink. Low-level access to the pattern matcher is possible by coding in C++, Scheme or Python.
- Six hard-coded indexes are kept by AtomSpace. User-defined indexes are not yet supported (due to lack of demand).
- AtomSpace contents may be viewed with several different visualisation tools including AtomSpace visualiser.
- AtomSpace benchmark tool measures the performance of a dozen basic atoms and AtomSpace operations. A diary in the benchmark directory maintains a historical log of measurements.

ing for different situations? Or, does it have to be based on a learning algorithm that develops knowledge representations as time progresses?

In OpenCog, they call this knowledge representation AtomSpace.

AtomSpace, the very basic. It is in AtomSpace that different compo-

nents of the intelligent system you have in mind are held, and this is also the query/reasoning engine that fetches data, manipulates it and performs reasoning on it. Effectively, AtomSpace is a kind of graph database, where graphs are representations of data and procedures. What this means is that graphs can be data, executable programs or data structures. AtomSpace could also have hypergraphs. The vertices and edges of the graphs are atoms, which essentially capture the notion of an atomic formula in mathematical logic.

How to use these to develop your system

What you have to do is use atoms to form small portions of the mind, which when put together form the complete system. You cannot code line-by-line per se, as the system is built for use by automation and machine learning engines. Automated sub-systems manipulate, query and inference over data or programs,

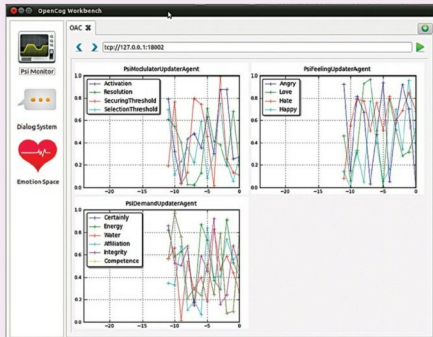


Fig. 2: An example of OpenCog workbench